The Corncob 3D Engine

This engine started with an experiment, and more code was added over time until it became a 3d game engine.

The most important structure in the engine is the obj3d structure. It has the 32bit xyz coord, the yaw pitch and roll angles, pointers to the 3D models, and many fields for various attributes and states of the object.

The some of the crucial fields for a 3D object are:

offset 24 - object type.

This is a word of attribute bits for the object. Some of these describe a quality of the object that shouldn't change over time, other bits are used to keep track of the state of this particular instance of the object.

Some examples

If type=0, then object is not drawn

if bit 0x8 == 1 then object is destructable

if bit 0x10 == 1 then object is a timed object, which gets removed when timer field hits 0

offset 72 - object template index

When it was time to add the ability to do in-game editing and from-file loading, it was necessary to be able to create new instances of objects without having them compiled into the code. Offset 72 is an index into 3 tables, each with about 60 items. These tables are in DEFOBJS.ASM:

objtimtable:

Each frame, each object calls the function in this table to do behavior stuff.

templatetable:

This table is used to dynamically create new objects. A copy of the object is created and added to the in-game list of current objects.

extendedtypetbl:

It was eventually realized that it was necessary to have certain attributes be fixed and not part of the state of the instance. The extendedtype table is just a list of 16bit integers with bit flags which are actually documented in a comment at the top of the list.

The coordinate system

Basically at the beginning, the 3d drawing functions were written in a coordinate system with 16 bit integers for position. However, this proved to be inadequate for a decent world size. If the coordinates covered the entire world, then the resolution of the 3d models would suffer.

So the model definitions are all in 16bit coordinates for x, y, z. However, the actual world uses 32bit coordinates for x, y and z. So every object in the world has a 32bit x, y, z coordinate, and a 16 bit roll, pitch, yaw angle. Before an object is drawn, a model is selected based on distance from the player (Level of Detail). A 9 element rotation matrix is created from the roll pitch and yaw and multiplied by the camera's rotation matrix, and the 32bit translation is done to get the view relative to the player's viewpoint.

Models are lists of primitive shapes. The primitives are:

NullObject (invisible undrawn objects)
Point Cloud a bunch of points, all the same color
Sphere a bunch of spheres- each can be a different color
WireFrame - a bunch of multiple-segment lines, each can be a different color
Polygon - an n-sided polygon of one color

Scene Management

Originally, the code tried to keep objects in various tables. Objects close to the player (camera) were kept in the near table. As objects got far away they were moved to the med-near, med-far, and far-far tables. This way, only objects in the near table needed to be checked for interaction with the player or checked for collisions.

Temporary objects such as bullets and explosion pieces were stored in their own list. The source file STRTEXP.ASM contained the code to manage the list. Also in this module is all the code to run the behaviors of timed objects. The objects in this list were kept in a different segment than the regular game objects. The array of words is called 'exps' and is defined in STARTEXP.ASM. The two tables neartbl and medtbl then became arrays of indices into the exps array.

The exps array contains 320 possible objects.

Objects which are farther away are kept in the memseg array which is in a different segment, I.E. in 'far' memory. There 884 (nmaxmedfar) items in the memseg array. Objects stored here are in the same structure as those in the exps array.

In order to enable a much larger world, there is yet another system. The world is divided into 64 square tiles arranged in a grid of 8x8 tiles. At each point in the world, the user can possibly see and interact with objects in the nearest 4 tiles. As the user moves, eventually objects become so far away that a tile which was once one of the four closest is no longer close enough. When this happens, objects in memory must be saved into this tile, and that tile is then compressed and stored in far memory. These objects cannot be accessed, drawn or interacted with until that time again becomes one of the four closest to the player.

So when the player (camera) moves such that some tiles become out of range, and new ones come into range the objects which belong in the old tile must be removed from the memseg table and compressed into the tile memory. Then the new tile must have its objects decompressed and added to the memseg table.

note:

In many of the comments, there are references to objects being 'bullets'. Many of these comments are now incorrect, since the original array of objects kept in the default segment. What happened was that the memory in the segment reserved for temporary objects became the storage for all active objects.